Lab Project Report

Name: Muhammad Saad, Muhammad Abdullah Siddiqui	ID: ms10054, ms10545	Section: T1

Introduction

Throughout the Computer Architecture Labs, we first learned how to write assembly codes, and later on we began designing key modules such as Register File, Arithmetic Logic Unit, Data Memory, Instruction Memory, Immediate Generator, and Main and ALU Control (Control Unit) etc., to help us run the assembly codes we learned to write initially on a processor we built ourselves. These key modules were the building blocks of our processor.

In the last lab, all the modules created before were integrated together to form a single cycle processor where we ran our assembly codes by passing machine codes as inputs. In this Lab Project, our goal was to run a countdown timer assembly code on our processor and then extend its functionality to support new instructions that were not previously implemented.

Task A

The first task was to run a countdown timer assembly program written in Lab 10 on our processor.

Steps Taken:

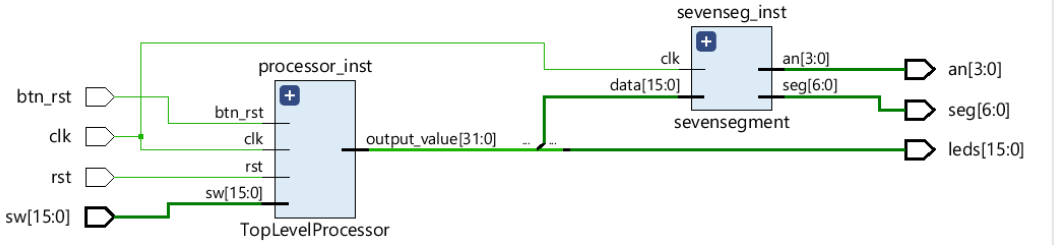
1. Developed, updated, and verified all key modules such a Immediate Generator and Control Unit especially to add two new instructions, jal and jalr, to our processor to support the assembly program. These were tested in this code instead of Task C.
2. Integrated all updated modules together to form a top level processor.
3. Also designed a seven segment module to display outputs to FPGA
4. Created a topModule to integrate the top level processor with the seven segment design, where lower 16 bits of the output from the processor would be displayed on LEDs and the upper 16 bits were passed into the seven segment design.
5. Created a testbench to test the processor
6. Loaded the assembly program using a .mem file containing the machine codes into the Instruction Memory.

7. Wrote constraint files for our design to run synthesis and implementation and then tested our outputs on an FPGA and verified the results
8. Designed a testbench and ran simulation to test the output of our program using waveforms.
9. Verified the program execution using waveforms that showed expected results

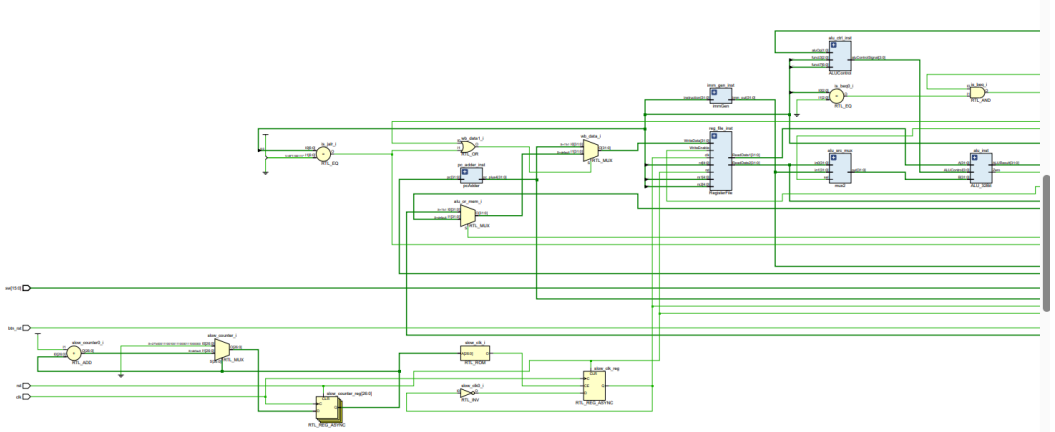
The processor was successful in executing the program as correct results were verified using FPGA and waveforms.

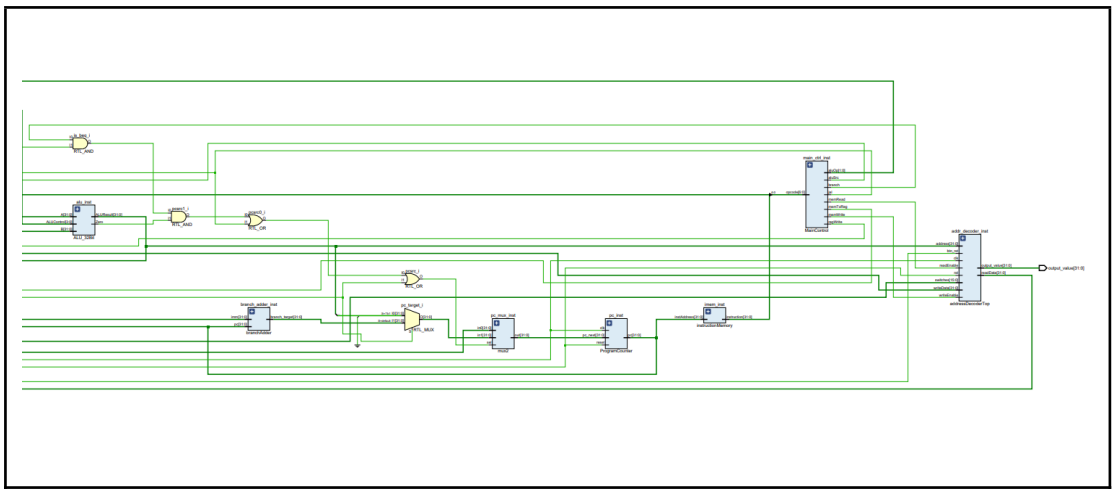
Schematic:

Top Module:

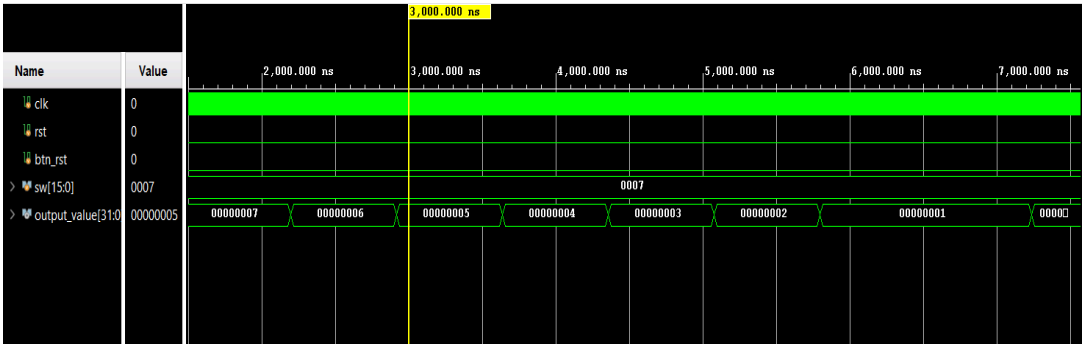


TopLevelProcessor:

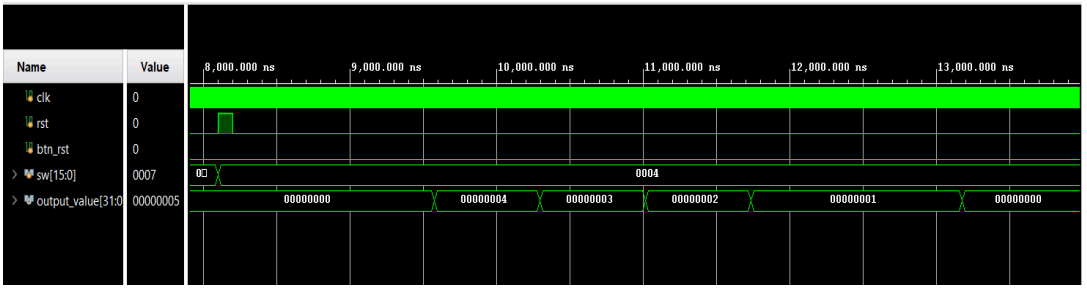




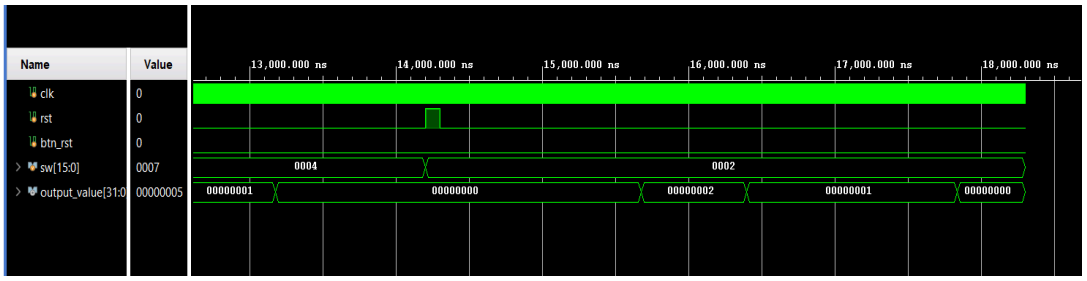
Waveforms:



Test case #1: input 7 (111), we can see a countdown happening from 7->6->5->4->3->2->1->0



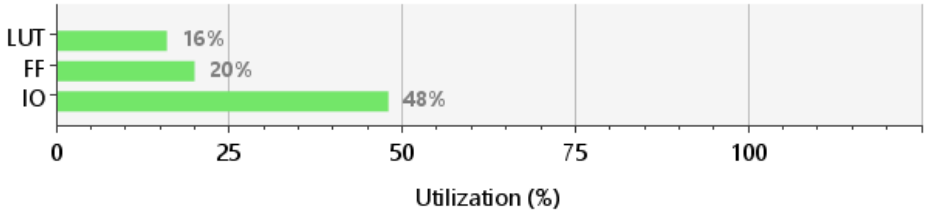
Test case #2: same behaviour with 4



Test case #3: same behaviour with 2

Utilization Reports:

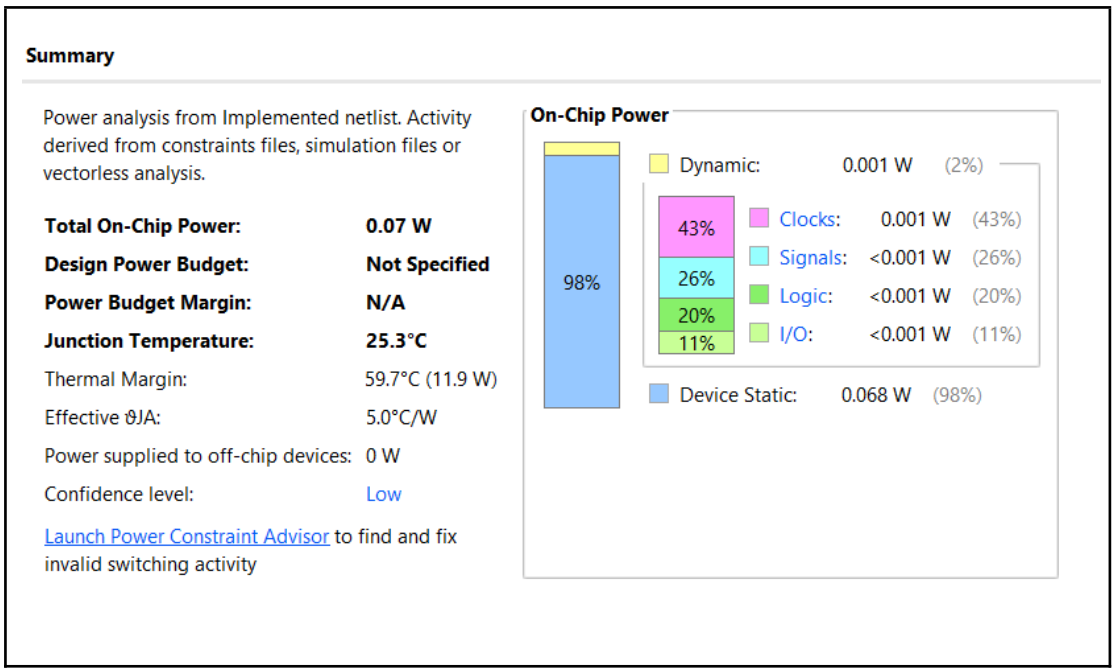
Resource	Utilization	Available	Utilization %
LUT	3430	20800	16.49
FF	8508	41600	20.45
IO	51	106	48.11



Design Timing Summary

Setup		Hold		Pulse Width	
Worst Negative Slack (WNS):	5.694 ns	Worst Hold Slack (WHS):	0.297 ns	Worst Pulse Width Slack (WPWS):	4.500 ns
Total Negative Slack (TNS):	0.000 ns	Total Hold Slack (THS):	0.000 ns	Total Pulse Width Negative Slack (TPWS):	0.000 ns
Number of Failing Endpoints:	0	Number of Failing Endpoints:	0	Number of Failing Endpoints:	0
Total Number of Endpoints:	28	Total Number of Endpoints:	28	Total Number of Endpoints:	29

All user specified timing constraints are met.



Task B

After implementing 2 new instructions, JAL and JALR in Task A, 2 new instructions were implemented, LUI and ANDI. I will be explaining JALR, LUI, and ANDI as my 3 new instructions implemented for this Lab Project as BNE was rejected).

JALR:

It was implemented in Task A and performs an indirect jump by calculating the target address as rs1 + immediate. The least significant bit of the target address is forced to 0. At the same time, PC + 4 is written back into the destination register rd, so the processor can return after a jump.

Datapath modifications:

- Added logic to detect the JALR opcode 1100111.
- Used the ALU to calculate the jump target address.
- Added logic to force the least significant bit of the ALU result to zero.
- Updated the PC selection path so that the processor can choose the JALR target address instead of the normal PC + 4.
- Updated the write-back path so that PC + 4 can be written into rd.
- Ensured JALR works with the existing register file, ALU, immediate generator, and program counter.

Control Path changes:

- Modified the control unit to recognize the JALR opcode.
- Set RegWrite = 1 because JALR writes PC + 4 into rd.
- Set ALUSrc = 1 because the second ALU input comes from the immediate value.
- Set the ALU operation to ADD so the ALU calculates rs1 + immediate
- Added is_jalr logic in the top module to select the correct next PC.
- Updated the PC control condition so that JALR can change the program counter.

This instruction was tested in Task A.

LUI:

This instruction loads a 20-bit immediate value into the upper 20 bits of the destination register and fills the lower 12 bits with zeros.

Datapath modifications:

- Added immediate generation logic for the U-type instruction format.
- Updated the immediate generator so that instruction bits [31:12] are shifted left by 12 bits.
- Added a new write-back path so the generated immediate value can be written directly into the register file.
- Updated the write-back multiplexer to select the immediate value for LUI.
- Ensured LUI does not disturb the existing ALU, memory, branch, or jump datapath.

Control Path changes:

- Modified the control unit to recognize the LUI opcode 0110111.
- Added a new control signal called lui.
- Set RegWrite = 1 because LUI writes the immediate value into rd.
- Set memory read and memory write signals to 0 because LUI does not access data memory.
- Updated the write-back selection logic so that when lui = 1, the register file receives the immediate value instead of the ALU or memory output.

Results were verified by designing a testbench and observing waveforms, and also via FPGA demonstration

ANDI:

This instruction performs a bitwise AND operation between a register value and an immediate value.

Datapath modifications:

- Used the existing ALU datapath to perform the AND operation.
- Updated the immediate generator to produce the correct I-type immediate for ANDI.
- Updated the ALU input multiplexer so that the second ALU input comes from the immediate value.
- No new memory path was required because ANDI only operates on register and immediate data.
- Ensured the result from the ALU is routed back to the register file through the normal write-back path.

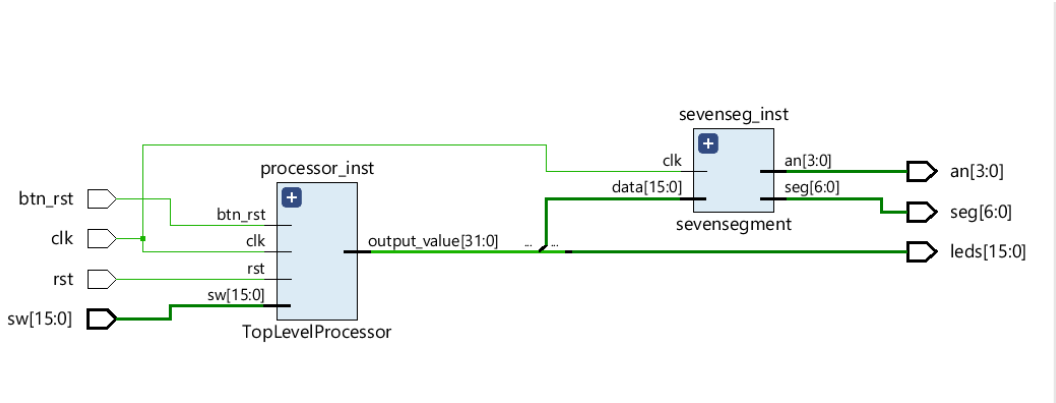
Control Path changes:

- Modified the control unit to recognize I-type arithmetic instructions using opcode 0010011.
- Set RegWrite = 1 because ANDI writes the ALU result into rd.
- Set ALUSrc = 1 because the second ALU operand is an immediate value.
- Set MemRead = 0 and MemWrite = 0 because ANDI does not use data memory.
- Updated the ALU control unit to decode funct3 = 111 as an AND operation.
- Generated the correct ALU control signal for bitwise AND.

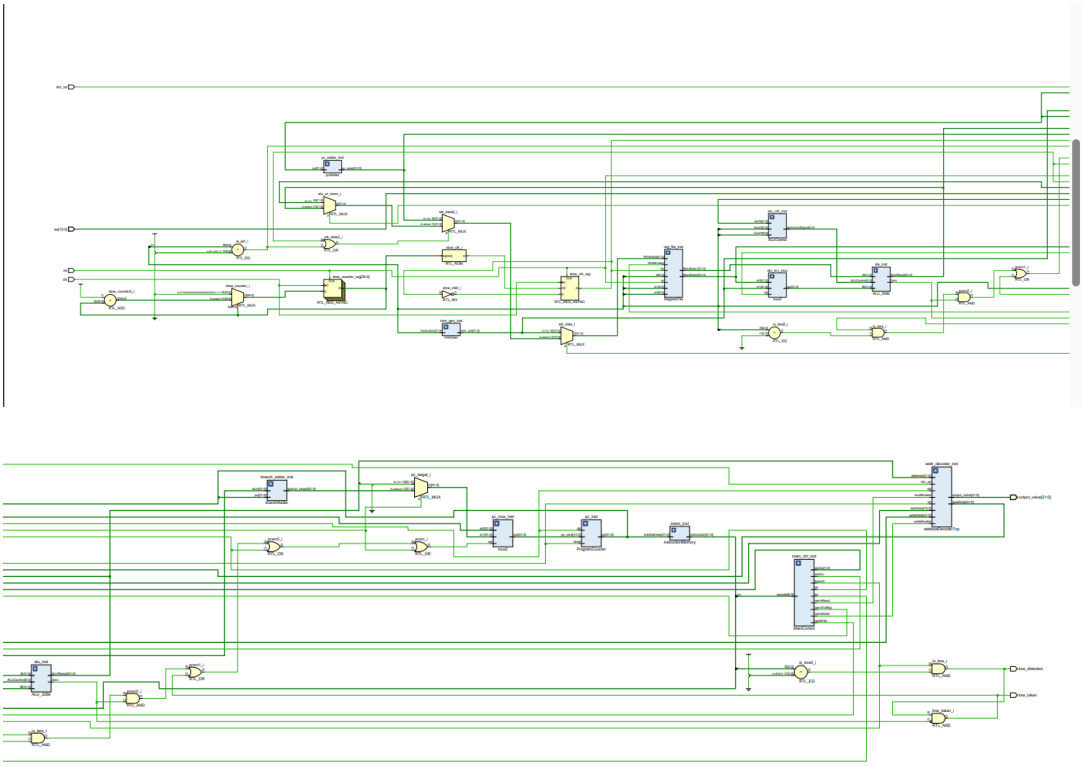
Results were verified by designing a testbench and observing waveforms, and also via FPGA demonstration

Schematic:

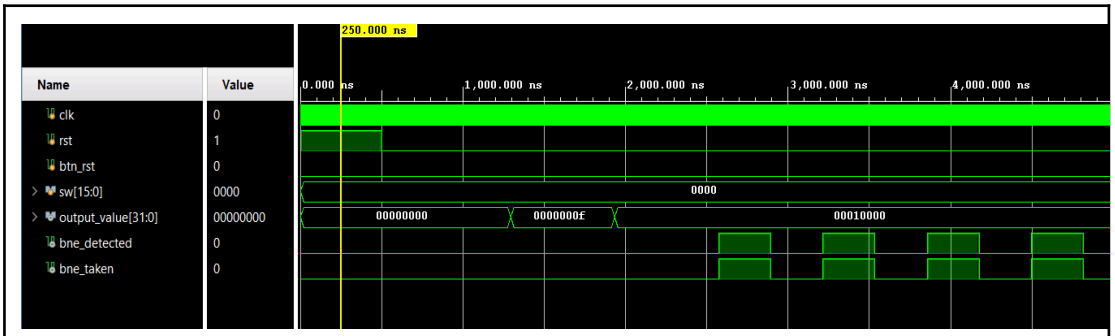
Top Module:



Top Level Processor:



Waveforms:

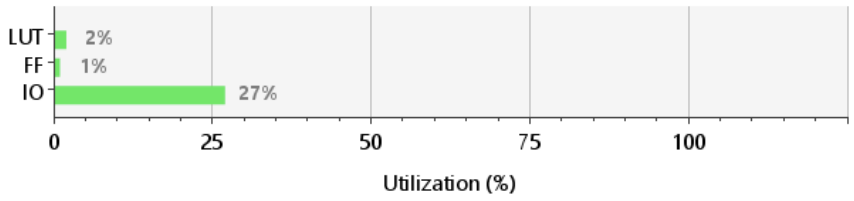


Our assembly code first used andi x5, x5, 15, where x5 = 255, so the result was 15, which is shown in the waveforms.
Next, we did lui x5, 0x00010, so that 0x00010000 was loaded into x5, which can be seen from the waveforms.
jalr was already tested in Task A.

Utilization Reports:

Summary

Resource	Utilization	Available	Utilization %
LUT	440	20800	2.12
FF	178	41600	0.43
IO	29	106	27.36



Design Timing Summary

Setup		Hold		Pulse Width	
Worst Negative Slack (WNS):	5.313 ns	Worst Hold Slack (WHS):	0.287 ns	Worst Pulse Width Slack (WPWS):	4.500 ns
Total Negative Slack (TNS):	0.000 ns	Total Hold Slack (THS):	0.000 ns	Total Pulse Width Negative Slack (TPWS):	0.000 ns
Number of Failing Endpoints:	0	Number of Failing Endpoints:	0	Number of Failing Endpoints:	0
Total Number of Endpoints:	63	Total Number of Endpoints:	63	Total Number of Endpoints:	48

All user specified timing constraints are met.

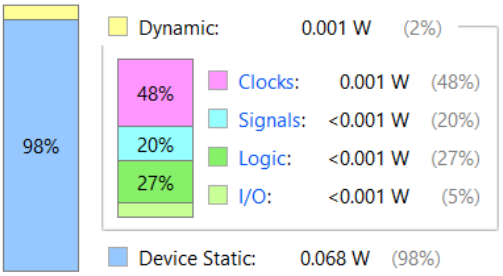
Summary

Power analysis from Implemented netlist. Activity derived from constraints files, simulation files or vectorless analysis.

Total On-Chip Power:	**0.07 W**
Design Power Budget:	**Not Specified**
Power Budget Margin:	**N/A**
Junction Temperature:	**25.3°C**
Thermal Margin:	59.7°C (11.9 W)
Effective ϑJA:	5.0°C/W
Power supplied to off-chip devices:	0 W
Confidence level:	Low

Launch Power Constraint Advisor to find and fix invalid switching activity

On-Chip Power



Task C

In this task, we wrote an assembly program that calculates the factorial of a number and then adds a number 0x00010000 to the output which is loaded using LUI. Also, it uses andi x5, x5, 15 on the register that reads the switches so that only the last 4 switches are read to avoid very large values and display 0x00010000 on FPGA properly.

Steps Taken:

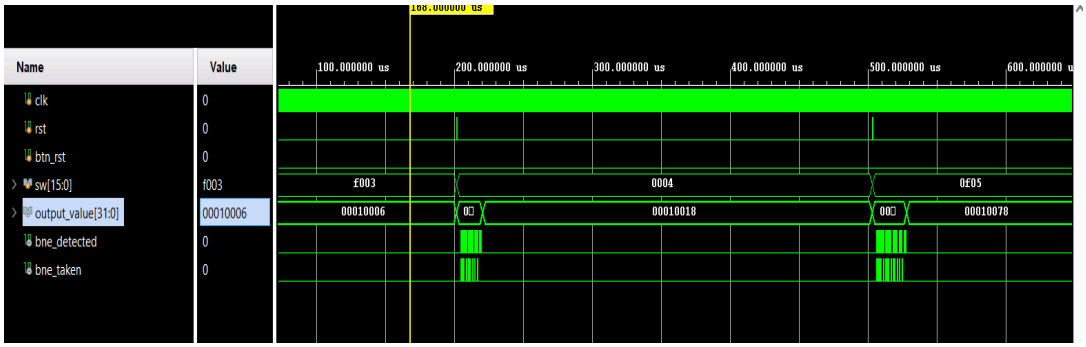
1. Created another .mem file with the machine codes of this assembly program and passed that into the Instruction Memory.
2. Used the updated processor from Task B which has the new instructions we are testing
3. Created a constraint file to run synthesis and implementation and then tested the outputs on an FPGA
4. Further verified outputs via testbench waveforms which showed correct results.

The processor successfully executed the program, including the newly added instruction, confirming correct integration.

Schematic:

Same as Task B

Waveforms:



As we can see, when the input is 0xf003, the upper 12 bits are ignored because of andi and input is taken as 0x0003, and 3! = 6 which is added to 0x00010000 to get 0x00010006 as output.

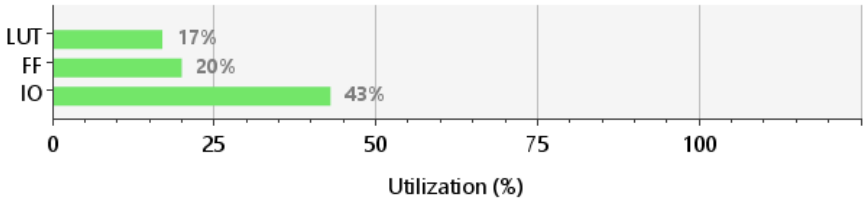
The next input is 0x0004, which is simply 4! + 0x00010000 = 24 + 0x00010000 = 0x18 + 0x00010000 = 0x00010018 as output.

The last input is 0x0f05, where the upper bits are ignored and input is taken as 0x0005, so we get 5! + 0x00010000 = 120 + 0x00010000 = 0x78 + 0x00010000 = 0x00010078 as output.

Utilization Reports:

Summary

Resource	Utilization	Available	Utilization %
LUT	3562	20800	17.13
FF	8495	41600	20.42
IO	46	106	43.40



Design Timing Summary

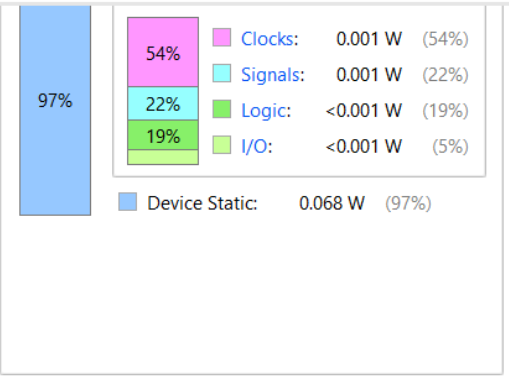
Setup		Hold		Pulse Width	
Worst Negative Slack (WNS):	5.423 ns	Worst Hold Slack (WHS):	0.235 ns	Worst Pulse Width Slack (WPWS):	4.500 ns
Total Negative Slack (TNS):	0.000 ns	Total Hold Slack (THS):	0.000 ns	Total Pulse Width Negative Slack (TPWS):	0.000 ns
Number of Failing Endpoints:	0	Number of Failing Endpoints:	0	Number of Failing Endpoints:	0
Total Number of Endpoints:	63	Total Number of Endpoints:	63	Total Number of Endpoints:	48

All user specified timing constraints are met.

Summary

Total On-Chip Power:	**0.071 W**
Design Power Budget:	**Not Specified**
Power Budget Margin:	**N/A**
Junction Temperature:	**25.4°C**
Thermal Margin:	59.6°C (11.9 W)
Effective ϑJA:	5.0°C/W
Power supplied to off-chip devices:	0 W
Confidence level:	Low

Launch Power Constraint Advisor to find and fix invalid switching activity



Conclusion:

During our work on this project, we designed and implemented a single cycle processor by integrating the modules mentioned above. We then tested the countdown timer assembly program on it and then implemented new instructions that our modified processor supports. We tested these new instructions using waveforms and FPGA demonstrations. Overall, this project provided us with a clear understanding of how all building blocks of a processor come together and how assembly code instructions are executed on a processor.